

Theory and Methodology

The multicovering problem

Nicholas G. Hall

The Ohio State University, Columbus, OH 43210, USA

Dorit S. Hochbaum *

University of California, Berkeley, CA 94720, USA

Received June 1989; revised February 1990

Abstract: The multicovering problem is: Min CX subject to $AX \geq b$, $X \in \{0, 1\}$, where A is a matrix whose elements are all zero or one and b is a vector of positive integers: this problem has many applications in scheduling, location and business areas. A variety of industrial, commercial and military contexts give rise to large problems of this kind, frequently at or beyond the limits of current computational tractability. Our research goal is the development of a special-purpose algorithm capable of solving large problems of this kind. Conceptual and computational inputs to the algorithm arise from three sources: from the generalization of computational tools developed for more specific problems (in particular the set covering problem), from the adaptation of general integer programming and combinatorial optimization methods, and from the development within this paper of special techniques designed to take advantage of the structure of the problem studied. Following a description of the algorithm we give some computational results for randomly generated problems.

Keywords: Integer programming; set covering; heuristics; cutting planes

1. Introduction

1.1. Applications and overview

A large variety of scheduling, location, business and other problems can be expressed as a mathematical model called the *multicovering* or *generalized set covering problem*. Among these are many problems which can be modeled by the set covering problem and which can alternatively be modeled, where unreliability is a factor, by the multicovering problem, the structure of which can be used to enforce coverage more than once.

Examples of applications where multiple coverage can be important include: location of emergency service facilities, radio and other communications systems, military applications, various marketing applications, crew scheduling, security and independent checking. We now discuss each of these briefly.

Regarding the location of emergency service facilities, a higher level of service is provided if there are two fire stations within a short distance of a building at risk than if there is only one. A recent application of the multicovering problem, for a radio communications system in West Germany, was undertaken by van Slyke (1981). The problem was formulated by van Slyke so that the variables (columns) represented repeaters, capable of relaying messages, whose locations were to

* The research of this author was supported in part by NSF grant ECS850-1988 and by the Office of Naval Research under grant ONR N00014-91-J-1241.

be selected from a finite set of possible locations so as to maintain communication between a fixed set of terminals. A military application was also discussed by van Slyke, with regard to the problem of 'terrain masking', where some areas of terrain are hidden by other: from airborne radar surveillance. An interesting marketing application occurs in the area of product design. Frequently a product, particularly one whose development and launching are likely to be expensive undertakings, may be designed to appeal to many individually identifiable target markets. One design problem is that of minimizing the product cost subject to the provision of a number of features attractive to each target market (i.e., for each row with this formulation).

The multicovering problem can be applied to crew scheduling problems for which the decision variables can take only values of zero or one, yet requirements can be general integers. One example would be the staffing of a project deep underground or underwater, or in hazardous circumstances such as radioactivity, intense heat or cold, where safety precautions, or the capacity of transport vehicles (e.g., diving bell), prohibit the simultaneous start or termination of work by more than one crew. The multicovering problem could also be used in the optimal design of checks in an accounting system so as to cover each aspect of the system a specified number of times. More coverage could be required for areas where the greatest potential loss could occur.

1.2. Mathematical statement of the problem

The multicovering problem (MCP) is

$$\begin{array}{ll} \text{Min} & CX \\ \text{s.t.} & AX \geq b, \\ & X \in \{0, 1\}, \end{array}$$

where A is an $m \times n$ -matrix whose entries are all zero or one, and b is a vector of positive integers.

The MCP is closely related to the set covering problem (SCP), which has a similar formulation in which b denotes a vector of ones and where A is a matrix whose entries are all zero or one. An extensive bibliography for the SCP was given by Balas (1981). The SCP is NP-hard (Karp, 1972). The MCP is a generalization of that problem, hence the MCP is NP-hard. As such we do not

expect that an efficient polynomial algorithm that solves the MCP exists. The purpose of this paper is to develop an algorithm for finding the optimal solution to MCP (or a solution close to it) that is efficient for certain classes of problem instances and for problem inputs of moderate size.

1.3. Related literature

The only thorough computational study of the MCP of which the authors are aware is that of van Slyke (1981). An earlier preliminary study of Wets and van Slyke (Network Analysis Corporation, 1974) was responsible for the development of four primal heuristics for the MCP: two of these were tested and the results of this testing appear in Section 2. A comprehensive bibliography for the SCP and the set partitioning problem (where equality constraints replace inequalities in the SCP) can be found in Balas (1982). There have been several major computational studies of this problem. Lemke, Salkin and Spielberg (1971) described an implicit enumeration approach using linear programming at subproblems. Following this, Etcheberry (1977) was the first to apply Lagrangian relaxation to this class of problems.

Balas and Ho (1980) developed an algorithm for the SCP which incorporated the use of Lagrangian relaxation in a branch-and-bound approach. They also used a primal 'greedy' heuristic for which worst-case performance had been evaluated by Johnson (1974) and Lovász (1975) for the unweighted case and Chvátal (1979) for the weighted case. Paixao (1984) used decomposition and state space relaxation to solve large instances of the set covering problem. Beasley (1987) combined dual ascent, subgradient optimization, and linear programming on a large vector processing computer to solve instances with up to 400 constraints and 4000 variables.

One heuristic approach to the SCP which seems to perform well is the LP heuristic described by Hochbaum (1980a, 1980b). This involves identifying the feasible dual solution to the linear programming relaxation of the SCP and the inclusion of variables in the cover only when the corresponding dual constraint is binding. Heuristics for the MCP were outlined and analyzed by Dobson (1982), Hochbaum (1980b), and Hall and Hochbaum (1986). We note that there is

at present no worst-case analysis for the variations of the greedy heuristic used by van Slyke.

The method of Lagrangian relaxation for obtaining strong lower bounds has been used successfully by Held and Karp (1970, 1971) for the traveling salesman problem. The theoretical justification for this approach was described by Held, Wolfe and Crowder (1974) and by Geoffrion (1974). Other applications have included location problems by Cornuejols, Fisher and Nemhauser (1977), cluster analysis by Mulvey and Crowder (1979), data base location by Fisher and Hochbaum (1980) and the SCP by Etcheberry (1977), by Balas and Ho (1980) and by Hey (1980). Attempts to improve the ability of the subgradient optimization method to converge have centered around solving the problem of 'zigzagging' by using more than one direction vector to obtain smoother convergence (Crowder, 1974; Camerini, Fratta and Maffioli, 1975).

1.4. Overall solution approach

We will describe in detail the various elements in a branch-and-bound approach for solving instances of the MCP. Section 2 contains a description of primal heuristics for finding feasible solutions. In Section 3 we describe heuristics which are used to find dual feasible solutions giving lower bounds on optimal cost. These bounds are greatly strengthened by the use of Lagrangian relaxation with subgradient optimization (Section 5). Section 4 develops disjunctive cutting planes which are used to eliminate solutions which cannot be optimal. Details of the overall enumeration procedure appear in Section 6.

2. Primal heuristics

2.1. The development of primal heuristics

It was decided early in this study to develop and test a number of different primal heuristics. We introduce a new type of heuristic and estimate that in at least 90% of the randomly problems studied, the heuristics of this type or the reduced-cost heuristic that evolved from the LP heuristic of Hochbaum (1980a, 1980b) are the source of the ultimately optimal solution. We report the results of testing in Section 2.2 below.

The first heuristic tested was the extension by Dobson (1982) of the greedy heuristic often used for the SCP (see Johnson, 1974; Lovász, 1975; Chvátal, 1979). However, its performance was disappointing; other heuristics that were devised outperformed it. All the later heuristics use more information than greedy and appear to benefit therefrom. Some of them take longer to run, however.

Five of the heuristics tested are of the 'greedy' type, as now illustrated.

Example 2.1. Consider the following set covering problem

$$\begin{aligned} \min \quad & 3X(1) + 4X(2) + 5X(3) + 6X(4) \\ & X(1) + X(2) + X(3) \geq 1, \\ & X(2) + X(3) + X(4) \geq 1, \\ & X(3) + X(4) \geq 1, \end{aligned}$$

$$X(j) = 0 \text{ or } 1, \quad j = 1, \dots, 4.$$

We set the value of incorporating variable $X(j)$ to be

$$f(j) = \sum_{i=1}^m a(i, j)/C(j),$$

$$\text{giving } f(1) = \frac{1}{3}, f(2) = \frac{1}{2}, f(3) = \frac{3}{5}, f(4) = \frac{1}{3}.$$

Since its value is largest, variable index 3 is chosen and it constitutes a cover of cost 5. The overall algorithm contains one heuristic which runs about as fast as the greedy heuristic, and two which run somewhat slower.

In their study of primal heuristics for the SCP, Balas and Ho (1980) tried different criteria which alter the balance of influence between the cost coefficient and the number of rows which the column covers: heuristics 2, 3, 4 and 5 in our study are based on a similar idea. An alternative functional form, which appears to produce good results (see Tables 2 and 3), is

$$\frac{1}{C(j)} \ln \left(\frac{R}{(R - W(j))} \right)$$

where

R = Weight of the rows which still need to be covered.

$W(j)$ = Weight of the rows which variable $X(j)$ can cover.

$C(j)$ = Cost of variable $X(j)$.

The performance of greedy heuristics and those with the modified greedy rule is similar when

$W(j)/R$ is small, since $\ln [R/(R - W(j))]/C(j) \rightarrow W(j)/C(j)$. As above, it is possible to define 'weight' in various ways and several different ideas were tried. Table 1 shows the selection criteria which we tested.

2.2. The testing of primal heuristics

We report on the testing of the ten heuristics whose criteria appear in Table 1. Testing was

Table 1
Summary of primal Heuristics ^a

Heuristic nr.	
1	$\frac{1}{C(j)} \sum_{i \in L} a(i, j)$
2	$\frac{1}{C(j)} \sum_{i \in L} \frac{b(i)a(i, j)}{rsum(i)}$
3	$\frac{1}{C(j)} \sum_{i \in L} \frac{a(i, j)}{space(i)}$
4	$\frac{1}{C(j)} \sum_{i \in L} a(i, j)b(i)$
5	$\frac{1}{C(j)} \sum_{i \in L} \frac{b(i)a(i, j)}{space(i)}$
6	$\frac{1}{C(j)} \ln \frac{\sum_{i \in L} \frac{b(i)}{rsum(i)}}{\sum_{i \in L} \frac{b(i)}{rsum(i)} - \sum_{i \in L} \frac{b(i)a(i, j)}{rsum(i)}}$
7	$\frac{1}{C(j)} \ln \frac{\sum_{i \in L} \frac{b(i)}{rsum(i)}}{\sum_{i \in L} \frac{b(i)}{rsum(i)} - \sum_{i \in L} \frac{a(i, j)}{rsum(i)}}$
8	$\frac{1}{C(j)} \ln \frac{\sum_{i \in L} \frac{b(i)}{space(i)}}{\sum_{i \in L} \frac{b(i)}{space(i)} - \sum_{i \in L} \frac{b(i)a(i, j)}{space(i)}}$
9	$\frac{1}{C(j)} \ln \frac{\sum_{i \in L} 1}{\sum_{i \in L} 1 - \sum_{i \in L} a(i, b)}$
10	$\frac{1}{C(j)} \ln \frac{\sum_{i \in L} b(i)}{\sum_{i \in L} b(i) - \sum_{i \in L} a(i, j)}$

^a Variables are selected which MAXimize these criteria.
 $L = \{i, \text{s.t. } b(i) > 0\}$.
 $rsum(i) = \sum_{j=1}^n a(i, j)$, $space(i) = rsum(i) - b(i)$.
 Criteria 6–10, when denominators equal zero, use $(\text{Max}_j \{C(j)\} + \text{Min}_j \{C(j)\}) / (2d) \cdot (1/C(j)) \cdot \sum_{i \in L} a(i, j)$, where d is the density of the matrix A .

performed on 25 randomly generated problems ranging in size from 30 rows $\times$ 30 columns with a uniform requirement of 2, to 100×200 with a uniform requirement of 30. We used problems with uniform requirements, because such problems are intuitively the hardest to solve since the dual variables may be less easily distinguishable than for problems with varying requirements. Our early computational experience supported this intuition.

Table 2 gives the results of testing, where the figures shown are percentage errors compared to the best heuristic value obtained. These results are summarized in Table 3. The heuristics chosen at this stage for inclusion in the algorithm were numbers 3, 5, 6 and 8. In testing of the complete algorithm, however, heuristic 3 performed less well than the other three and was removed. In addition to the ten heuristics already described, we implemented another heuristic (see Section 5) that uses reduced costs as input. Because heuristics 5 took significantly less time to run than either heuristic 6 or heuristic 8 (it ran about 4 times faster on average), it was used more frequently than the others.

3. Dual heuristics

3.1. Objectives

Early in a solution process, before subgradient optimization is used for the first time, dual heuristics provide a vector of dual variables which acts as an initial solution for the subgradient process. The dual heuristics also provide a dual feasible solution which is used to generate cutting planes (Section 4).

3.2. The Dual Greedy Heuristic

The dual problem associated with the linear programming relaxation of the MCP is:

$$\begin{aligned}
 \text{Max} \quad & \sum_{i=1}^m b(i) \cdot u(i) - \sum_{j=1}^n v(j) \\
 \text{s.t.} \quad & \sum_{i=1}^m a(i, j) \cdot u(i) - v(j) \leq C(j), \\
 & j = 1, \dots, n \\
 & u(i) \geq 0, \quad i = 1, \dots, m, \\
 & v(j) \geq 0, \quad j = 1, \dots, n.
 \end{aligned} \tag{3.1}$$

Table 2
The MCP

Heuristic nr.	Percentage errors above best heuristic value									
	1	2	3	4	5	6	7	8	9	10
1	7.9	0.2	* ^a	0.7	0.2	0.7	0.7	0.6	0.8	0.7
2	2.7	*	0.3	2.7	*	2.7	0.3	0.3	0.8	2.7
3	4.3	*	4.3	10.7	*	2.6	1.7	0.3	1.4	10.7
4	6.1	1.1	1.1	0.2	1.1	*	4.9	1.7	0.2	0.2
5	2.0	1.9	0.8	2.3	4.0	*	4.7	3.3	2.3	2.3
6	2.8	1.4	1.4	1.7	*	2.9	3.3	1.5	1.2	1.2
7	7.8	*	*	*	*	0.1	0.1	3.2	0.9	3.0
8	8.4	5.6	4.2	2.3	5.6	1.6	2.5	1.6	*	*
9	*	*	*	*	*	1.6	*	1.6	*	*
10	3.9	1.7	5.5	5.5	1.7	*	2.5	*	2.4	2.3
11	4.9	2.1	*	3.5	*	*	*	*	3.5	3.5
12	10.0	1.5	0.3	3.6	*	0.7	0.6	*	1.5	1.5
13	10.4	*	4.9	4.6	*	1.6	3.1	1.6	2.6	4.6
14	*	1.2	*	2.0	3.3	1.5	*	1.5	3.8	3.8
15	2.4	0.7	1.2	2.4	*	1.0	4.3	4.7	6.1	5.1
16	18.8	1.7	1.6	0.1	1.7	5.3	6.2	*	0.4	0.1
17	5.1	2.7	3.4	1.9	1.6	1.4	2.4	*	1.9	1.9
18	8.6	0.6	1.1	0.6	1.1	*	3.3	0.4	3.3	3.3
19	9.7	0.5	2.7	3.0	*	0.3	4.6	1.6	2.6	2.0
20	9.4	*	1.6	3.1	3.3	*	1.9	1.1	3.3	3.8
21	10.8	6.3	*	4.5	2.1	3.1	8.0	0.8	5.1	5.4
22	8.3	4.0	*	2.6	2.0	4.0	4.9	4.0	2.6	2.6
23	4.1	1.9	2.0	2.2	3.7	1.9	*	1.3	2.2	3.2
24	7.1	0.4	4.0	3.7	1.3	1.4	2.6	*	3.8	3.8
25	3.1	1.5	1.8	0.8	2.0	1.8	0.6	*	0.8	0.8

^a * denotes the best heuristic value.

Let $J = \{1, \dots, n\}$ be the set of variable indices, $I = \{1, \dots, m\}$ be the set of row indices, and $PS \subseteq J$ be the cover found by the most recent primal heuristic application.

Let X be the characteristic vector of PS , namely

$$X(j) = \begin{cases} 1 & \text{if } j \in PS, \\ 0, & \text{otherwise,} \end{cases}$$

and let T be the set of 'tight' rows for PS , i.e.,

$$T = \left\{ i \in I \mid \sum_{j \in J} a(i, j) X(j) = b(i) \right\}.$$

Given the cover PS we derive a 'corresponding' dual solution in the sense that if a dual variable $u(i)$ is positive, then row i is in T (tight). This property makes our primal-dual heuristic satisfy one of the two sets of complementary slackness conditions:

$$\sum_{i \in I} u(i) \cdot \left(b(i) - \sum_{j \in J} a(i, j) X(j) \right) = 0.$$

The Dual Greedy Heuristic, described below, provides a dual feasible solution by assigning values to dual variables. The maximum value which would not violate feasibility is assigned.

Table 3
The MCP

Heuristic #	Number of problems solved as best heuristic									
	1	2	3	4	5	6	7	8	9	10
Best or equal	2	6	7	2	10	6	4	7	2	2
Best alone	0	0	3	0	3	3	1	4	0	0

Dual Greedy Heuristic (PS)

Given

PS = the set of variables in the cover,

T = the set of tight (primal) constraints;

Step 1. Initialize

$u(i) = 0$ for all $i \in I$,

$v(j) = 0$ for all $j \in J$,

$\bar{C}(j) = C(j)$ for all $j \in J$.

Step 2. Let $i \in T$

Step 2a. $\bar{C}_j(i) = \text{Min}_{j \in J - \text{PS}} \{C(j) | a(i, j) = 1\}$,

Step 2b. Update $u(i) \leftarrow u(i) + \bar{C}_j(i)$,

Step 2c. For all $j \in J$ such that $a(i, j) = 1$,
 $\bar{C}(j) \leftarrow \bar{C}(j) - \bar{C}_j(i)$.

Step 3. Set $T \leftarrow T - \{i\}$. If $T = \emptyset$, go to Step 4, otherwise go to Step 2.

Step 4. For all $j \in J$ such that $\bar{C}(j) < 0$ set
 $v(j) = -\bar{C}(j)$.

Set $s(j) = \bar{C}(j) + v(j)$ for all $j \in J$.

Output $s(\text{PS}) = \{s(j) | j \in J\}$ [heuristic reduced costs vector] and $(U_{\text{PS}}, V_{\text{PS}}) = \{u(i), i = 1, \dots, m, v(j), j = 1, \dots, n\}$.

Halt.

Since the choice of row $i \in T$ in Step 2 is arbitrary, we tested several selection rules.

I: Choose a row i with minimum potential to be covered, i.e., with minimum $\sum_{j=1}^n a(i, j) - b(i)$.

II: Choose row i that is potentially most costly to cover, i.e., with maximum $b(i) \cdot \bar{C}_j(i)$.

III: arbitrary choice.

The third rule of arbitrary choice produced poor results using both the criteria now considered. The best results in terms of maximizing the value of the lower bound obtained were given by rule II. However, the rule which minimized the number of variables in the cover with strictly positive heuristic reduced costs was rule I: this property is important for the cutting plane subroutine and that rule is therefore used.

Lemma 3.1. *The solution $u(i)$, $i = 1, \dots, m$, $v(j)$, $j = 1, \dots, n$, delivered by the Dual Greedy Heuristic, is dual feasible, i.e., it satisfies constraints (3.1).*

Proof.

I. We first prove $\sum_{i=1}^m a(i, j) u(i) - v(j) \leq C(j)$, $j = 1, \dots, n$.

Entering Step 4 of the Dual Greedy Heuristic,

$$\bar{C}(j) = C(j) - \sum_{i=1}^m a(i, j) u(i).$$

Case 1. If $\bar{C}(j) \geq 0$, from Step 4, $v(j) = 0$, and therefore

$$C(j) \geq \sum_{i=1}^m a(i, j) u(i) = \sum_{i=1}^m a(i, j) u(i) - v(j).$$

Case 2. If $\bar{C}(j) < 0$, from Step 4, $v(j) = -\bar{C}(j)$ and

$$v(j) + \bar{C}(j) = C(j) - \sum_{i=1}^m a(i, j) u(i) + v(j) = 0,$$

and therefore

$$C(j) = \sum_{i=1}^m a(i, j) u(i) - v(j),$$

satisfying the constraint.

II. We now prove $u(i) \geq 0$, $i = 1, \dots, m$.

Initially $u(i) = 0$, for all $i \in I$, from Step 1. From Step 2b, it is sufficient to show that $\bar{C}_j(i) \geq 0$ for chosen $i \in T$, all $j \in J - \text{PS}$, at each iteration k of Step 2 of the heuristic. From Step 2a, it is sufficient to show that $\bar{C}(j) \geq 0$ for all $j \in J - \text{PS}$ at each iteration k . From step 1, $\bar{C}(j) \geq 0$ for all $j \in J - \text{PS}$ at iteration 1. Assume $\bar{C}(j) \geq 0$ for all $j \in J - \text{PS}$ after $k - 1$ iterations but not after k iterations. Then from Step 2c in iteration k , $\bar{C}_j(i) > \text{Min}\{\bar{C}(j) | a(i, j) = 1\}$, which contradicts Step 2a.

III. Finally we prove $v(j) \geq 0$, $j = 1, \dots, n$. Entering Step 4, $v(j) = 0$, $j = 1, \dots, n$. In Step 4, a positive value is assigned to $v(j)$ or its value is unchanged. $\square$

4. Cutting planes**4.1. Objectives**

The aim in this section is to develop valid cutting planes of the form $\sum X(j) \geq 1$ for the problem

$$\begin{aligned} (\text{IP}) \quad & AX \geq b \\ & \sum C(j) X(j) \leq \beta, \\ & X \in \{0, 1\}. \end{aligned}$$

4.2. The generation of cutting planes

Let

$$N(i) = \{j | a(i, j) = 1\} \quad \text{for } i \in I$$

where $I = \{1, \dots, m\}$. We find inequalities $\sum_{j \in S(i)} X(j) \geq 1$ for $S(i) \subseteq N(i)$ such that the linear program

$$\begin{aligned} \min \quad & CX \\ \text{s.t.} \quad & AX \geq b, \\ & \sum_{j \in S(i)} X(j) \geq 1, \quad i \in I, \\ & 0 \leq X(j) \leq 1, \end{aligned}$$

has a dual feasible, and hence optimal, value $> \beta$, where β is defined in the formulation (IP) above. It follows that any feasible solution to (IP) must satisfy

$$X(j) = 0 \quad \text{for } j \in S(i)$$

for some $i \in I$. In other words,

$$\sum_{j \in N(i) \setminus S(i)} X(j) \geq b(i)$$

for some $i \in I$, or

$$\sum_{j \in Q(i)} X(j) \geq 1 \quad \text{for some } i \in I$$

where $Q(i)$ is any subset of $N(i) \setminus S(i)$ of cardinality $|N(i) \setminus S(i)| - [b(i) - 1]$. Finally we obtain the valid inequality

$$\sum_{j \in \bigcup_{i=1}^m Q(i)} X(j) \geq 1.$$

Example 4.1. Assume we have a constraint $X(2) + X(4) + X(5) \geq 1$ and a primal feasible solution PS which is $X(1) = X(2) = X(3) = 1$. Consider the added inequality $X(2) \geq 1$. If this inequality is not satisfied by any improved solution, then $X(2) = 0$ in all such solutions, which implies $X(4) + X(5) \geq 1$, since the constraint is tight. The last inequality is a cutting plane in that it is not satisfied by the current primal solution.

Example 4.2. Now assume that we have two constraints $X(2) + X(4) + X(5) \geq 1$, and $X(1) + X(6) \geq 1$, and PS as in Example 4.1. Further assume that if we add both the inequalities $X(2) \geq 1$ and $X(1) \geq 1$ to the problem, a dual feasible solution of value $\geq Z_f$ is easily found, where Z_f is the cost of the best known primal feasible solution. Then, from duality, any primal feasible solution of value

$< Z_f$ must violate *at least one* of those inequalities.

If it violates $X(2) \geq 1$, then $X(2) = 0$, which in turn implies (from the first constraint) that $X(4) + X(5) \geq 1$. Alternatively, if it violates $X(1) \geq 1$, then (from the second constraint) $X(6) \geq 1$. Since we know that at least one of the inequalities $X(4) + X(5) \geq 1$ and $X(6) \geq 1$ is satisfied by any solution of value less than Z_f , such a solution definitely satisfies $X(4) + X(5) + X(6) \geq 1$, the new cutting plane.

We now provide the cutting plane generation algorithm. Step 0 describes the input to the algorithm and initializes a set W , which will contain the indices of variables in the new cutting plane. In Step 1 we choose a variable index $j' \in R$ for construction of the added inequality. The inequality is specified by that choice, as well as by the choice in Step 2 of a tight constraint, and a dual variable with a nonnegative value $s(j')$ is attached to the inequality. This value is chosen such that it maintains a feasible dual solution to the enlarged problem. This increases the value of the dual feasible solution. It is empirically advantageous to include many variables in the inequality, because the resulting cutting plane is stronger. To see this, if we use $X(2) + X(4) \geq 1$ as the added inequality in the solution, and if that inequality is not satisfied by later solutions, then $X(2) = X(4) = 0$, which implies $X(5) \geq 1$. The cutting plane $X(5) \geq 1$ is stronger than $X(4) + X(5) \geq 1$, in that it removes more solutions.

Step 3 tests whether variables can be included in the inequality without violating dual feasibility. Step 4 tests whether the dual feasible value for the enlarged problem is less than Z_f . If so, additional inequalities are generated. If not, the algorithm terminates with a new cutting plane. For the cutting plane to be valid, the dual feasible value has to be at least as great as Z_f (see Lemma 4.2).

Cutting Plane Algorithm

Step 0 [Input and initialization].

Given a nonredundant primal feasible solution, PS, of cost P and a corresponding dual feasible solution (U_{PS}, V_{PS}) from the dual heuristic of value D . Given a vector of heuristic reduced costs $s(PC) = (s(1), \dots, s(n))$ corresponding to that dual feasible solution. As in Lemma

4.1, R represents the set of variables having value one in PS and also having strictly positive heuristic reduced costs, and T represents the set of constraints which are tight under PS.

Step 0.

Let $r = 0$, $W = \emptyset$.

Step 1. [Select a variable in R].

$r = r + 1$.

Let $j' \in R$.

Step 2. [Choose a tight row in which j' appears, that will imply a cutting plane].

Let $\bar{i} \in T$ such that $a(\bar{i}, j') = 1$.

[Note that since PS is a nonredundant solution such a $\bar{i} \in T$ can always be found.]

Step 3. [Testing for dual feasibility and specifying a new inequality].

For $j \in J - (\text{PS} \cup W)$ such that $a(\bar{i}, j) = 1$ do

if $s(j) < s(j')$, then $W = W \cup \{j\}$.

[In this case, including $X(j)$ in the added inequality would violate dual feasibility. Since $X(j)$ is not in the added inequality, it appears in the cutting plane derived from constraint $\bar{i}$.]

if $s(j) \geq s(j')$, set $s(j) = s(j) - s(j')$.

[Here $X(j)$ appears in the added inequality to which a dual variable of value $s(j')$ is attached. $s(j)$ measures the remaining slack in the dual constraint for column j .]

end

Step 4. [Add the new inequality and attach to it a dual variable with value $s(j')$. The enlarged problem is the original enlarged problem with this inequality added. Then compare the value of the dual feasible solution to the enlarged problem to the best primal solution cost, Z_f .]

Set $R = R - \{j'\}$

Set $D = D + s(j')$

If $D \geq Z_f$, go to Step 5. [From Lemma 4.1, we know this condition will be satisfied eventually, because only one variable index is removed from R at each iteration of Steps 1 to 4.]

If $D < Z_f$, go to Step 1.

Step 5.

Output the new cutting plane

$\sum_{j \in W} X(j) \geq 1$.

Terminate.

Lemma 4.2. *The cutting plane derived by the cutting plane algorithm is a valid cutting plane, in the sense that*

(a) *it eliminates the current solution PS;*

(b) *it is satisfied by any solution with cost strictly less than Z_f , the best feasible solution value discovered so far.*

Proof. (a) The set of variables, W , in each cutting plane inequality includes no variables from PS according to Step 2 of the algorithm. The overall cutting plane requires at least one variable from W to have value one.

(b) When the algorithm terminates after (say) r iterations of Steps 1 to 4, we have at hand a dual feasible solution (of value $D \geq Z_f$) to an enlarged problem. From weak duality, any primal solution for the enlarged problem has value no less than D . Thus any primal solution of value less than Z_f to the original problem must violate *at least one* of the new inequalities. Violation of inequality i is equivalent to a cutting plane of the form $\sum_{j \in W_i} X(j) \geq 1$, where $W = \bigcup_{i=1}^r W_i$, therefore any solution of value less than Z_f satisfies $\sum_{j \in W} X(j) \geq 1$. $\square$

We developed empirically successful heuristic rules for replacing the arbitrary choice of $j' \in R$ at Step 1 and of $\bar{i} \in T$ at Step 2, as we now state and discuss. Step 1' will replace Step 1 and Step 2' will replace Step 2 in the implementation of the algorithm.

Step 1'.

$r = r + 1$.

Let j' be such that

$$s(j') = \text{Min} \left\{ \text{Max}_{j \in R} s(j), \right. \\ \left. \text{Min}_{j \in R} [s(j) \mid s(j) + D \geq Z_f] \right\}.$$

Step 2'.

For all $t \in T$ such that $a(t, j) = 1$ do

rowadd(t) = $|\{j \in J - (\text{PS} \cup W) \text{ such that } a(t, j) = 1, s(j) < s(j')\}|$

rowin(t) = $\sum_{j \in R} a(t, j)$

score(t) = rowadd(t)/rowin(t)

end

Let $\bar{i}$ be such that score($\bar{i}$) = $\text{Min}_{t \in T} \text{score}(t)$.

The heuristic rules in Step 1' and Step 2' have been designed and empirically tested for the production of strong cutting planes. Step 1' chooses the variable index with largest reduced cost in an

attempt to minimize the number of iterations required. Step 2' chooses among rows according to how much W will increase in size using row t , compared to the number of iterations at which it may be possible to use row t .

The following example illustrates the algorithm and concludes this section. We use X_j in place of $X(j)$ for ease of presentation.

Example 4.3.

$$\begin{aligned} \text{Min} \quad & 6X_1 + 10X_2 + 4X_3 + 5X_4 + 6X_5 + 5X_6 + 3X_7 \\ \text{s.t.} \quad & X_1 + X_4 + X_7 \geq 1, \\ & X_1 + X_2 + X_3 + X_5 + X_6 + X_7 \geq 3, \\ & X_1 + X_2 + X_3 + X_4 + X_5 \geq 1. \end{aligned}$$

Let the output of the primal heuristic be: $PS = \{5, 6, 7\}$, $P = 14$, $T = \{1, 2, 3\}$. Assume this is also the best feasible solution found so far, i.e. $Z_f = 14$.

The corresponding dual heuristic solution is: $u(1) = 5$, $u(2) = 1$, $u(3) = 0$, $v(7) = 3$, $D = 5$, giving a heuristic reduced cost vector of $(0, 9, 3, 0, 5, 4, 0)$. In this case, $R = \{5, 6\}$.

In Step 1, we select $j' = 5$.

In Step 2, the tight constraints in which the variable X_5 appears are 2 and 3. To select the tight row from which to derive the added inequality and the cutting plane, we apply Step 2': $\text{rowadd}(2) = 2$, $\text{rowin}(2) = 2$, therefore $\text{score}(2) = 1$; $\text{rowadd}(3) = 3$, $\text{rowin}(3) = 1$, therefore $\text{score}(3) = 3$.

Selecting the row $\bar{i} = 2$, in Step 3 we find $s(1) = 0 - 5 < 0$, so $W = \{1\}$, $s(2) = 9 - 5 = 4$; $s(3) = 3 - 5 < 0$, so $W = \{1, 3\}$, and the inequality $X_2 + X_5 \geq 1$ is added to the problem. Note that if this inequality is to be violated by another solution, then that solution satisfies $X_1 + X_3 \geq 1$, in order to have the original problem constraint 2 satisfied.

In Step 4, $R = R - \{5\}$, $D = 5 + 5 = 10 < 14$.

Returning to Step 1, $R = \{6\}$, therefore $j' = 6$.

In Step 2, the only tight constraint for X_6 is constraint 2.

In Step 3, only $s(2) = 4 - 4 = 0$, that is, again $W = \{1, 3\}$, and the constraint $X_2 + X_6 \geq 1$ is added to the problem. Finally in Step 4, $D = 10 + 4 = 14$, so we can terminate and, using $W = \{1, 3\}$, output the new cutting plane constraint

$$X(1) + X(3) \geq 1.$$

We show below the enlarged problem, i.e., the original problem with the additional inequalities.

$$\begin{aligned} \text{Min} \quad & 6X_1 + 10X_2 + 4X_3 + 5X_4 + 6X_5 + 5X_6 + 3X_7 \quad \text{Dual} \\ & \text{variables} \\ \text{s.t.} \quad & X_1 + X_4 + X_7 \geq 1, \quad 5 \\ & X_1 + X_2 + X_3 + X_5 + X_6 + X_7 \geq 3, \quad 1 \\ & X_1 + X_2 + X_3 + X_4 + X_5 \geq 1, \quad 0 \\ & X_2 + X_5 \geq 1, \quad 5 \\ & X_2 + X_6 \geq 1, \quad 4 \end{aligned}$$

Note that the addition of the new inequalities has created a dual feasible solution whose value equals that of Z_f , i.e., 14.

5. Lagrangian relaxation

5.1. Objectives

We use Lagrangian relaxation to deliver improved lower bounds on the value of a problem or subproblem. In addition, the reduced costs output by the subgradient optimization process are used in Section 5.6 to construct a cover (feasible solution) for the problem.

5.2. A relaxation for the MCP

We seek to establish a strong lower bound on the problem value for the MCP. The method we use involves the relaxation of all the covering constraints. We then attempt to find as accurately as possible:

$$\text{Max}_{u \geq 0} L(u)$$

where

$$L(u) = \min_{0 \leq X \leq 1} [CX + u(b - AX)]. \quad (5.1)$$

The initial vector of dual variables, u^0 , is taken from the Dual Greedy Heuristic of Section 3. The updating of the vector u is done iteratively using subgradient ascent, namely:

$$u^{k+1} = \text{Max}\{u^k + t^k d^k, 0\} \quad (5.2)$$

where t^k is a scalar step size and d^k is a subgradient specified by

$$d^k = b - AX^k, \quad (5.3)$$

X^k being the solution vector for which the value of $L(u^k)$ is obtained.

For the step size t^k , we define Z_f = value of best known feasible solution, λ = a scalar parameter, and we set (as proposed in Held, Wolfe and Crowder, 1974)

$$t^k = \lambda \frac{Z_f - L(u^k)}{\|d^k\|^2}. \quad (5.4)$$

Solving the problem specified by u^k is an easy task. The problem can be written as $u^{k_b} + \text{Min}_{0 \leq x \leq 1} (C - u^k A)X$. The solution to this problem is

$$X^k(j) = 0 \quad \text{if } C(j) > \sum_{i=1}^m u^k(i)a(i, j),$$

$$X^k(j) = 1 \quad \text{if } C(j) < \sum_{i=1}^m u^k(i)a(i, j),$$

$$X^k(j) = 0 \quad \text{or } 1 \quad \text{if } C(j) = \sum_{i=1}^m u^k(i)a(i, j).$$

In 20 out of 30 instances tested, $X^k(j) = 1$ produced the faster convergence.

5.3. Finding a suitable step size

Initially, the scalar constant λ is set to 2 (empirical results showed that a value of 3 produced only slightly inferior results). In the subgradient optimization procedure this constant is reset, at any node, only by the following rule:

If no 'progress' has been made (i.e. no improved lower bound obtained) during the last p iterations of the subgradient procedure, λ is halved.

In the context of this rule, 'progress' is defined as the improvement which closes the gap between the previously best known upper and lower bounds by at least 5%. If very small improvements are considered as 'progress' for this purpose, the step size will often remain too high. We tested various settings for the value of p : $p = n$, $[n/3]$, $[n/6]$, m , $[m/3]$, $[m/6]$, where $[r]$ is the greatest integer less than or equal to r , and then fine tuned these values by adding a constant between one and ten. A setting of $p = [m/3] + 4$ worked best.

In the branch-and-bound tree we have a redefined problem at each new node. We examined

various possible ways of adjusting the step size constant at each new node. We begin with the value of the step size constant at the parent node, i.e., the node from which the new node is created. Then one of the following steps is performed.

1. Multiply the step size constant by 2 or 4 following the redefinition.

2. Restore the step size constant to a fixed level, for example, 1, following the redefinition.

3. Reset step size constant to $2\lambda^{1/4}$, where λ represents the value of the step size constant at subgradient termination for the parent node (but not to exceed 2).

Empirically, method 3 significantly outperforms the others, as measured by the average number of steps to convergence, so we use it in the branch-and-bound procedure.

5.4. Finding a suitable direction of improvement

One of the possible problems of subgradient optimization can be a tendency for convergence to take place over a zigzagging and therefore much slower path. A zigzagging path occurs when the values of individual dual variables fail to converge monotonically but instead fluctuate erratically. Identifying such behavior is important in that it can greatly reduce the speed of convergence. Crowder (1974) suggested the use of a subgradient made up of a linear combination of the current implied subgradient and the previous subgradient. A problem with this approach is finding a robust way of estimating the coefficients for the linear combination: no way has been found of achieving this for the MCP.

Camerini et al. (1975) tried to identify iterations where the subgradient implied by (5.3) forms a negative inner product with the previous subgradient, as would happen in circumstances of zigzagging. Five instances of the MCP, randomly generated as described in Section 7, were tested under the Camerini method and in no case was there any such evidence of zigzagging. A likely explanation of this result was that the step sizes used were too small. The experiment was therefore repeated with step size constants multiplied by a factor of 2, then 5, and eventually 10, but there was still no evidence of zigzagging even with these greatly increased step sizes.

5.5. Elimination of variables

Here we develop an easy test for the elimination of variables that results in significant computational improvement of the overall algorithm. Part of the output of the subgradient optimization method when it has been applied for k iterations is a set of reduced costs

$$\bar{C}_u^k(j) = C(j) - \sum_{i=1}^m u^k(i) a(i, j), \quad j = 1, \dots, n$$

where $u^k(i)$, $i = 1, \dots, m$ are the dual variables after k iterations. In the lemma and the proof which follows we use the notation $\bar{C}(\bar{j})$ in place of $\bar{C}_u^k(j)$, and we use u to denote the vector $u^k(i)$, $i = 1, \dots, m$. As before, Z_f represent the cost of the best known feasible solution. We start with a multicovering problem, with n unfixed variables. We denote the problem in which j variables are not fixed by $[P_j]$ (thus $n - j$ variables are fixed), and the value of the Lagrangian (5.1) given by the vector $u^k(i)$, $i = 1, \dots, m$, for $[P_j]$ is denoted $L_j(u)$.

Lemma 5.1. *Given a vector of multipliers u . If for some $\bar{j}$,*

$$\bar{C}(\bar{j}) \geq Z_f - L_n(u) \quad (5.5)$$

is satisfied, then variable $X(\bar{j})$ cannot appear with value one in any solution to $[P_n]$ of cost strictly less than Z_f .

Proof. Consider the smaller problem, $[P_{n-1}]$, created by setting $X(\bar{j}) = 1$. This problem can be found from $[P_n]$ by removing $X(\bar{j})$ and its cost $C(\bar{j})$, and also removing column $\bar{j}$ (which we call $A^{\bar{j}}$) from matrix A , and reducing the requirements vector according to the coefficients in $A^{\bar{j}}$. The precise formulation of this problem is

$$\begin{aligned} & [P_{n-1}] \\ & \text{Min} \quad \sum_{j \in J - \bar{j}} C(j) X(j) \\ & \text{s.t.} \quad \sum_{j \in J - \bar{j}} a(i, j) X(j) \geq b(i) - a(i, \bar{j}), \\ & \quad \quad i = 1, \dots, m, \\ & \quad \quad 0 \leq X(j) \leq 1, \quad X(j) \text{ integer}, \\ & \quad \quad 1 \leq j \leq n, \quad j \neq \bar{j}. \end{aligned}$$

Any solution to $[P_n]$ which has cost strictly less than Z_f , and has $X(\bar{j}) = 1$, must solve $[P_{n-1}]$ with cost strictly less than $Z_f - C(\bar{j})$. However, we will show that if the lemma condition (5.5) is met, the vector u provides a lower bound $L_{n-1}(u)$ on the cost of a solution to $[P_{n-1}]$ of value greater than $Z_f - C(\bar{j})$. This means that all solutions to $[P_n]$ of cost strictly less than Z_f have $X(\bar{j}) = 0$. We shall now prove that (5.5) implies

$$L_{n-1}(u) \geq Z_f - C(\bar{j}). \quad (5.6)$$

Recalling that $\bar{C}(\bar{j}) = C(\bar{j}) - \sum_{i=1}^m u(i) a(i, \bar{j})$, it suffices to show that

$$L_n(u) = L_{n-1}(u) + \sum_{i=1}^m u(i) a(i, \bar{j}).$$

We substitute the value of the optimal solution in the expressions for $L_n(u)$ and $L_{n-1}(u)$.

$$\begin{aligned} L_n(u) &= \sum_{0 \leq X(j) \leq 1} \left\{ \sum_{j \in J} C(j) X(j) \right. \\ &\quad \left. + \sum_{i=1}^m u(i) \left[b(i) - \sum_{j \in J} a(i, j) X(j) \right] \right\} \\ &= \sum_{i=1}^m u(i) b(i) \\ &\quad + \sum_{j \in J} \text{Min} \left\{ 0, C(j) - \sum_{i=1}^m u(i) a(i, j) \right\}, \end{aligned}$$

and

$$\begin{aligned} L_{n-1}(u) &= \text{Min}_{0 \leq X(j) \leq 1} \left\{ \sum_{j \in J} C(j) X(j) \right. \\ &\quad \left. + \sum_{i=1}^m u(i) \left[b(i) - a(i, \bar{j}) \right. \right. \\ &\quad \left. \left. - \sum_{j \in J - \bar{j}} a(i, j) X(j) \right] \right\} \\ &= \sum_{i=1}^m u(i) [b(i) - a(i, \bar{j})] \\ &\quad + \sum_{j \in J - \bar{j}} \text{Min} \left\{ 0, C(j) - \sum_{i=1}^m u(i) a(i, j) \right\}. \end{aligned}$$

Finally, since $\bar{C}(\bar{j}) \geq Z_f - L_n(u) \geq 0$, we can write

$$\begin{aligned}
 L_n(u) &= \sum_{i=1}^m u(i)b(i) \\
 &\quad + \sum_{j \in J-\bar{j}} \text{Min} \left\{ 0, C(j) - \sum_{i=1}^m u(i)a(i, j) \right\} \\
 &= \sum_{i=1}^m u(i)[b(i) - a(i, \bar{j})] \\
 &\quad + \sum_{j \in J-\bar{j}} \text{Min} \left\{ 0, C(j) - \sum_{i=1}^m u(i)a(i, j) \right\} \\
 &\quad + \sum_{i=1}^m u(i)a(i, \bar{j}) \\
 &= L_{n-1}(u) + \sum_{i=1}^m u(i)a(i, \bar{j}). \quad \square
 \end{aligned}$$

Lemma 5.1 enables us to remove variables which will not appear in an optimal solution, and doing so results in considerable computational saving. In most problems between 15 and 40% of the original variables are removed by the time of the first branching.

5.6. Constructing a cover from the reduced costs

We use the reduced costs $\bar{C}(j)$, $j = 1, \dots, n$, output by subgradient optimization as well as an incumbent cover to construct a new cover for the problem as shown below. The intuition for our heuristic approach is as follows. We expect that a number of variables in the incumbent cover may also be in the optimal solution. Variables from outside the incumbent cover are more likely to appear in an optimal cover if they have low reduced costs. We therefore construct a cover made up of the incumbent cover and additional variables, following which redundant variables are removed, in nonincreasing cost order. For randomly generated problems this heuristic approach has performed well, although its success is dependent upon the accuracy of the subgradient convergence. In testing on 50 problem instances, the heuristic cover had cost equal to or lower than the best primal heuristic from Table 1 in 39 instances, and strictly lower in 31 instances. We summarize the heuristic as follows:

Step 0. Given a set of variables constituting the best known cover C .

Step 1. Call subgradient procedure [C], for K_2 iterations or convergence within $\frac{1}{2}\%$, whichever occurs first. One output of this procedure is a set of reduced costs $\bar{C}(j)$, $j = 1, \dots, n$.

Step 2. If $\bar{C}(\bar{j}) \geq Z_f - L(u)$, eliminate variable $\bar{j}$ from the cover C (Lemma 5.1). If constraint i becomes unsatisfied, go to Step 3, otherwise to Step 4.

Step 3. Reconstruct a feasible cover which does not include $\bar{j}$ by adding to $C - \{\bar{j}\}$ one unfixed variable such that $a(i, j) = 1$, which has the lowest reduced cost among such variables. Repeat until a feasible cover is achieved.

Step 4. Add a number t (discussed below) of extra variables to the cover according to lowest reduced cost.

Step 5. Remove any redundancy from the cover in order of decreasing cost among variables in the newly constructed cover.

Empirically successful values of the parameter t , as tested on 50 problem instances, have ranged from 3 to 6: in general a higher value seems to be helpful when subgradient convergence is poor. In the overall algorithm we use a specification of 4 for this parameter.

6. Implicit enumeration

6.1. Introduction

When the techniques described in previous sections cannot make further improvements on best known upper and lower bounds, or such improvements occur slowly, then it becomes computationally efficient to divide the set of remaining feasible solutions and work with subproblems of the original problem.

6.2. Selecting a node to branch from

We need to identify which part of the enumeration tree to develop next. Various possibilities are

I. Worst bound – where nodes are likely to be hard to fathom.

II. best bound – the most likely source of an improved solution.

III. depth-first search (Tarjan, 1972).

IV. breadth-first search.

We tested these four strategies on 30 problem instances, generated as described in Section 7. In 23 of these instances, strategy II gave the quickest overall computation time. We thus use strategy II in the overall algorithm.

6.3. Specifying the branching direction

The following scheme was devised with the idea of limiting the choice of variable with largest reduced cost to a restricted set. This method created a more balanced enumeration tree, compared to the choice of variable with largest or smallest reduced cost. The method used was:

Step 1. At the node chosen to branch from (see Section 6.2), identify the row with the largest dual variable at termination of the application of subgradient optimization which gave the highest lowest bound.

Step 2. In the row identified at Step 1, identify the unfixed variables which have $a(i, j) = 1$.

Step 3. Among such variables choose the one with the largest reduced cost.

The row identified at Step 1 is likely to be one which is hard to cover (hence the high value of the dual variable) and the technique of selecting the variable with the largest reduced cost seems to give an element of balance to the tree.

6.4. Summary of the algorithm

The algorithm solves instances of the MCP by branch and bound. At each branching point one subset is created by the inclusion of a variable and another subset by its exclusion. Each subset constitutes a subproblem to which we apply the various subroutines now described.

The algorithm is initialized by setting the lower bound to 0 and the upper bound to plus infinity. After initialization, the algorithm uses a primal heuristic to obtain a feasible solution (the value of which is an upper bound) to the problem, and a dual heuristic to obtain a lower bound. The output of these heuristics is used to generate a new constraint (cutting plane) which eliminates the incumbent primal feasible solution, thereby reducing the size of the feasible region. Since any improved feasible solution will satisfy the new

constraint (as described in Section 4), the search for an optimal solution is hastened.

If at any time a primal or dual heuristic reduces the gap between the upper and lower bound to less than $\frac{1}{2}\%$, the current subproblem is eliminated. When the new cutting plane has been added, the next step is another primal heuristic application. When the number of iterations of the primal–dual cutting-plane sequence has reached K_1 (the value of which has been experimentally determined), then Lagrangian relaxation is used to obtain a stronger lower bound on the subproblem value.

Two possible results can occur at an application of Lagrangian relaxation: either the lower bound on the subproblem value increases to within $\frac{1}{2}\%$ of the upper bound (in which case the subproblem is eliminated) or the process is terminated after a number of iterations K_2 . In the former case, the node in the tree corresponding to the subproblem is fathomed and a new subproblem with best lower bound is considered. In the latter case, the next subroutine depends on the number of subgradient applications used at the current subproblem. If this number is less than a parameter K_3 , the next step is the use of a special primal heuristic using the reduced costs from Lagrangian relaxation (Section 5). If the number of iterations equals K_3 , the current subproblem is put aside and considered, among other unfathomed subproblems, for further partition. When all subproblems have been fathomed, the problem is solved and the lowest-value feasible solution is output, as well as the maximum relative error (not to exceed $\frac{1}{2}\%$).

Decision points (see Figure 1):

1 Is lower bound less than $[0.995]$ [upper bound]?

2 Does the number of primal–dual cutting-plane iterations (u) equal at least K_1 ?

3 Does the number of iterations of subgradient optimization (v) equal at least K_2 ?

4 Does the number of applications of subgradient optimization (to termination), w , equal at least K_3 ?

5 Do any unfathomed nodes exist?

Parameter specifications used were:

$K_1 = 6$.

$K_2 = 2m$ at the initial node, thereafter m , where m is the number of original constraints in the problem.

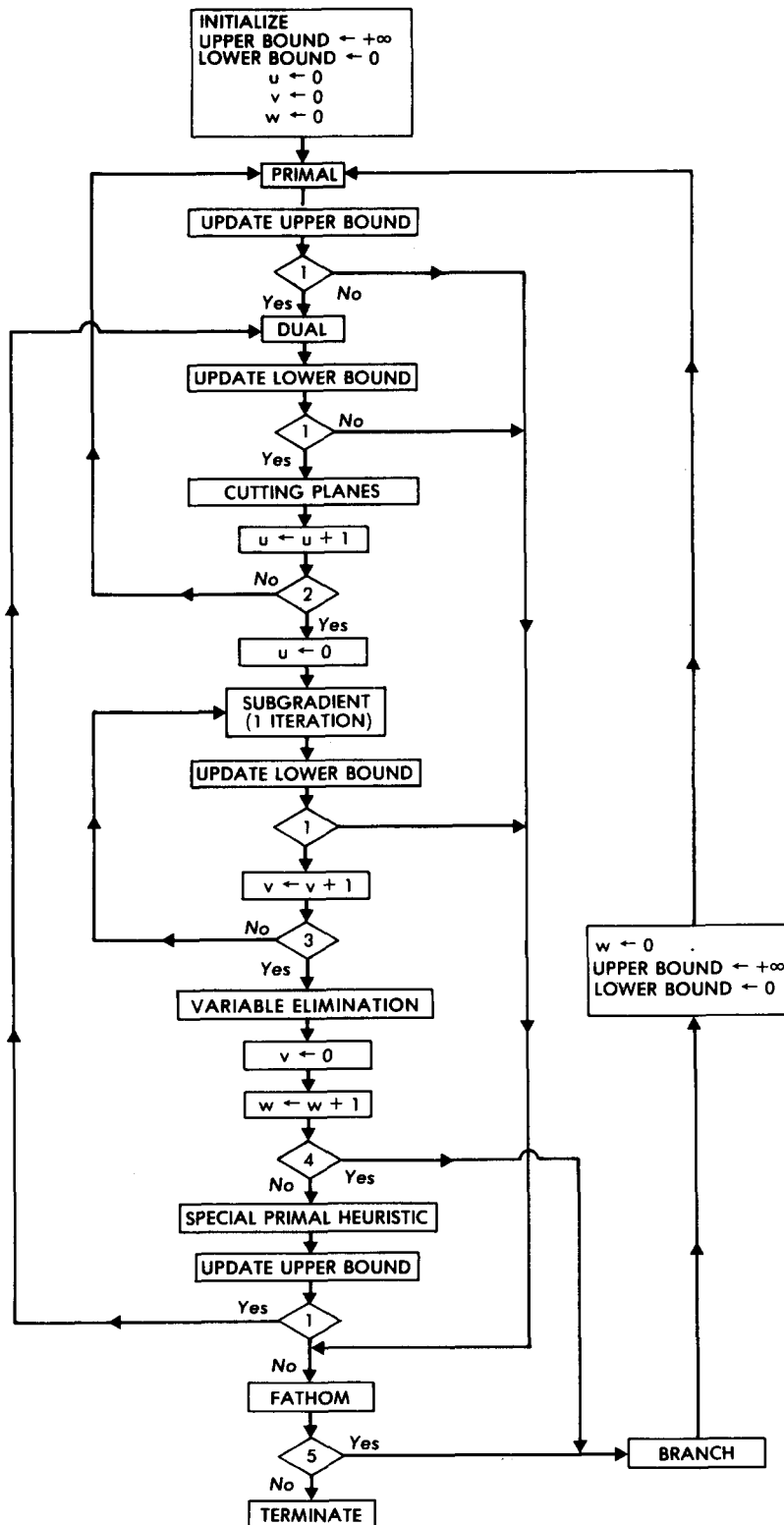


Figure 1. Flow chart of the algorithm

Table 4
Computational results (0.5% tolerance) ^a

Series #	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Number of rows	30	30	30	30	30	30	50	50	50	50	50	50	50	50
Number of columns	60	60	60	90	90	90	100	100	100	200	200	200	200	200
Matrix density	0.1	0.1	0.15	0.1	0.1	0.1	0.08	0.08	0.1	0.05	0.05	0.05	0.05	0.05
Right-hand side	2	1,3	1,3	3	2,4	1,5	1,3	2	1,3	3	2,4	2	1	1,5
Solved (of 20)	20	20	20	20	20	20	20	20	20	20	20	20	20	20
Best time	0.2	0.2	0.2	0.3	0.2	0.4	2.1	2.0	0.9	2.4	1.7	0.9	0.6	1.7
Mean time	0.7	0.6	1.1	2.0	1.7	2.3	5.5	8.0	7.0	15.2	13.5	11.1	2.6	12.8
Median time	0.5	0.6	0.9	1.8	1.5	1.7	4.8	5.5	5.8	14.0	11.9	7.1	1.6	9.5
Worst time	2.7	1.8	3.5	6.7	8.9	10.4	11.8	24.6	18.0	42.4	48.6	38.9	12.1	41.2

^a All times are in Amdahl 470 seconds. In series 2, 3, 5, 6, 7, 9, 11 and 14, right-hand-side requirements were randomly generated integers over the ranges shown.

$K_3 = [(m + n)/100] + 1$ at the initial node, thereafter 1, where n is the number of variables in the problem and $\lceil r \rceil$ is the smallest integer greater than or equal to r .

7. Computational results

The complete algorithm was programmed in Fortran and tested on the Amdahl 470 system of The Ohio State University. In Table 4 we present a summary of results for 280 randomly generated problems, ranging from 30 constraints and 60 variables to 50 constraints and 200 variables, in 14 series of 20 problems each. For each series the highest, lowest, median and mean computation times are shown. Problems were generated so that no constraints were trivial, i.e., in every case the matrix row sum exceeded the right-hand-side requirement. Variable costs were randomly generated in the range (1, 100). Series 1, 4, 8, 10, 12 and 13 had uniform requirements, whereas the other series had integer requirements randomly generated in the ranges shown.

All 280 problems tested were solved within a time limit of 60 CPU seconds, using a tolerance of $\frac{1}{2}\%$ (no solution exists more than $\frac{1}{2}\%$ better than that delivered by the algorithm). In general, computation times increased with problem size. Problems with nonuniform right-hand-side requirements seem easier to solve than those with uniform requirements, as can be seen in a comparison of the mean computation times for series 1, 4, 8 and 10 against those for series 2, 5, 6, 7, 11 and 14. On average over these 10 series, problems with uniform requirements took 6.5 seconds

to solve and those with nonuniform requirements took 5.3 seconds. This calculation is based on using the mean time for series 5 and 6 and for

Table 5
Computational efficiency at different tolerance levels

Problem #	Data series (see Table 4)	Computation time		
		0.5% Tolerance	1.0% Tolerance	2.0% Tolerance
1	1	1.01	0.91	0.77
2	1	1.36	1.06	1.00
3	1	0.75	0.75	0.46
4	1	0.31	0.19	0.14
5	1	0.79	0.71	0.59
6	3	0.21	0.17	0.17
7	3	3.00	2.22	1.96
8	3	1.71	1.29	1.13
9	3	0.92	0.85	0.66
10	3	0.33	0.31	0.31
11	4	3.14	1.70	0.76
12	4	1.90	1.11	0.92
13	4	3.92	3.05	1.79
14	4	0.47	0.47	0.29
15	4	0.83	0.69	0.51
16	6	2.16	1.69	1.24
17	6	0.75	0.59	0.52
18	6	5.24	3.77	3.41
19	6	2.45	1.93	1.70
20	6	0.91	0.71	0.53
21	9	8.73	4.99	4.99
22	9	12.90	12.15	9.73
23	9	1.64	1.36	0.97
24	9	5.08	3.78	2.08
25	9	2.51	2.40	2.23
26	10	12.75	10.17	8.03
27	10	2.54	2.12	2.02
28	10	9.90	7.07	5.58
29	10	18.16	16.22	4.51
30	10	6.90	5.33	4.14

series 11 and 14 for comparison with series 4 and 10, respectively. The effect of increasing matrix density can be seen in a comparison of series 2 and series 3 and also of series 7 and series 9. In each case a fairly small increase in matrix density increased the average running time significantly.

In Table 5 we present empirical results for the impact of using different tolerance levels on computation times. Thirty problems were generated, five in each of six of the series shown in Table 4, and the complete algorithm was applied to these problems using tolerance levels of, respectively, $\frac{1}{2}$, 1 and 2%. In 28 out of 30 problems (all except #6 and #10), significant reductions in computation time were achieved by increasing tolerance level from $\frac{1}{2}$ to 2%. Improvements in computation time tended to be more even (for 1 vs $\frac{1}{2}$ % as against 2 vs 1%) for larger problems. A likely reason for this is the greater number of near-optimal solutions which are available in a larger problem. The exact solution of problems in series 1 and 2 (respectively, 10 and 11) took about twice (respectively, five times) as long as solution within $\frac{1}{2}$ % tolerance.

To summarize, the algorithm seems to solve large randomly generated examples of the MCP very reliably. Among possible topics for future research is its performance on real-world problem data.

References

- Balas, E. (1980), "Cutting planes from conditional bounds: A new approach to set covering", *Mathematical Programming Study* 12, 19–36.
- Balas, E. (1982), "A class of location, distribution, and scheduling problems: Modeling and solution methods", *Revue Belge de Statistique, d'Informatique et de Recherche Opérationnelle* 22, 36–69.
- Balas, E., and Ho, A. (1980), "Set covering algorithms using cutting planes, heuristics and subgradient optimization: A computational study", *Mathematical Programming Study* 12, 37–60.
- Beasley, J.E. (1987), "An algorithm for set covering problems", *European Journal of Operational Research* 31, 85–93.
- Camerini, P.M., Fratta, L., and Maffioli, F. (1975), "On improving relaxation methods by modified gradient techniques", *Mathematical Programming Study* 3, 26–34.
- Chvátal, V. (1979), "A greedy heuristic for the set covering problem", *Mathematics of Operations Research* 4/3, 233–235.
- Cook, S.A. (1971), "The complexity of theorem-proving procedures", *Procedures of the 3rd ACM Symposium on the Theory of Computing*, 151–158.
- Cornuejols, G., Fisher, M.L., and Nemhauser, G.L. (1977), "Location of bank accounts to optimize float", *Management Science* 23/8, 789–810.
- Crowder, H.P. (1974), "Computational improvements for subgradient optimization", IBM Technical Report RC4907, Yorktown Heights, New York.
- Dobson, G. (1982), "Worst-case analysis of greedy heuristics for integer programming with non-negative data", *Mathematics of Operations Research* 7/4, 515–531.
- Etcheberry, J. (1977), "The set covering problem: A new implicit enumeration algorithm", *Operations Research* 25, 760–772.
- Fisher, M.L., and Hochbaum, D.S. (1980), "Database location in computer networks", *Journal of the ACM* 27/4, 718–735.
- Garey, M.R., and Johnson, D.S. (1979), *Computers and Intractability: A Guide to the Theory of NP-Completeness*, Freeman, San Francisco, CA.
- Geoffrion, A.M. (1974), "Lagrangian relaxation for integer programming", *Mathematical Programming Study* 2, 82–114.
- Hall, N.G., and Hochbaum, D.S. (1986), "A fast approximation algorithm for the multicovering problem", *Discrete Applied Mathematics* 15, 35–40.
- Held, M., and Karp, R.M. (1970), "The traveling salesman problem and minimum spanning trees", *Operations Research* 18, 1138–1162.
- Held, M., and Karp, R.M. (1971), "The traveling salesman problem and minimum spanning trees II", *Mathematical Programming* 1, 6–25.
- Held, M., Wolfe, P., and Crowder, H.P. (1974), "Validation of subgradient optimization", *Mathematical Programming* 6, 62–88.
- Hey, A.M. (1980), "Algorithms for the set covering problem", Ph.D. Thesis, Imperial College, University of London.
- Hochbaum, D.S. (1980a), "Approximation algorithms for the weighted set covering and node cover problems", Working Paper #64-79-80, Graduate School of Industrial Administration, Carnegie-Mellon University. Also, *SIAM Journal on Computing* 11/3 (1982) 555–556.
- Hochbaum, D.S. (1980b), "Fast approximation algorithms for some integer programming problems", Working Paper #21-80-81, Graduate School of Industrial Administration, Carnegie-Mellon University.
- Johnson, D.S. (1974), "Approximation algorithms for combinatorial problems", *Journal of Computer and System Sciences* 9, 256–278.
- Karp, R.M. (1972), "Reducibility among combinatorial problems", in: R.E. Miller and J.W. Thatcher (eds.), *Complexity of Computer Computations*, Plenum Press, New York, 85–103.
- Lemke, C.E., Salkin, H.M., and Spielberg, K. (1971), "Set covering by single branch enumeration with linear programming subproblems", *Operations Research* 19, 98–102.
- Lovász, L. (1975), "On the ratio of optimal integral and fractional covers", *Discrete Mathematics* 13, 383–390.
- Mulvey, J.M., and Crowder, H.P. (1979), "Cluster analysis: An application of Lagrangian relaxation", *Management Science* 25, 329–340.
- Network Analysis Corporation (1974), "The practical impact of recent computer advances in the design and analysis of

- large-scale networks”, Semiannual Technical Report (to DARPA), Great Neck, NY.
- Paixao, J. (1984), “Algorithms for large scale set covering problems”, Ph.D. Thesis, Department of Management Science, Imperial College, London.
- Papadimitriou, C.H., and Steiglitz, K. (1982), *Combinatorial Optimization*, Prentice-Hall, Englewood Cliffs, NJ.
- van Slyke, R. (1981), “Covering problems in CCCI Systems”, Report to the Air Force Office of Scientific Research.
- Tarjan, R.E. (1972), “Depth-first search and linear graph algorithms”, *SIAM Journal on Computing* 1/2, 146–159.